

Project plan — Île-de-France route planner

1. Module architecture

We split the code into four independent layers, each with a clear contract:

- `parser/` — reads raw GTFS .txt files (with pandas) and produces clean intermediate tables: stops, routes, trips, stop_times, transfers.
- `graph/` — turns those tables into a route-space graph stored in `networkx` (or a hand-rolled adjacency list if we want full control over performance experiments).
- `solver/` — implements Dijkstra and A*, exposing a single `shortest_path(graph, source, target)` API so they're swappable.
- `viz/` — matplotlib-based map rendering, takes a graph + path and produces the figure.
- `cli.py / main.py` — glues it together, takes two station names, returns an itinerary.

Keeping the parser ignorant of the graph and the solver ignorant of GTFS means switching to bus data later only touches `parser/`.

2. From GTFS to graph

GTFS gives us several CSV-like files. For the metro slice we only need five:

- `stops.txt` → station id, name, lat, lon.
- `routes.txt` → lines (we filter `route_type == 1` for metro; later 3 for bus, 2 for RER).
- `trips.txt` → each scheduled run of a line.
- `stop_times.txt` → arrival/departure at each stop on each trip — this is where our edge weights come from.
- `transfers.txt` → official minimum transfer times at interchange stations (great gift from the IDFM feed, we should honor it).

The parser's job is to produce three clean DataFrames: `stations`, `line_segments` (consecutive (stop_A, stop_B, line) with mean travel time), and `transfers` (stop_id, line_in, line_out, min_time).

3. The route-space graph design

This is the most important design decision and ties directly to how we handle interchanges. We use **one node per (station, line) pair** rather than one node per station. So Châtelet, which serves lines 1, 4, 7, 11, 14, becomes five distinct nodes.

Three kinds of edges live in the graph:

- **Riding edges** — between consecutive (station, line) nodes on the same line, weight = mean travel time between those stops.
- **Transfer edges** — between (station, line_A) and (station, line_B) at the *same* station, weight = transfer penalty (either a constant like 3 min, or `min_transfer_time` from `transfers.txt`).

- **Boarding/alighting edges** — a virtual "station hub" node per station, connected to each of its (station, line) nodes with weight 0. The user queries hub→hub, so the algorithm naturally chooses which line to start and end on.

Why bother with this structure? Because it makes transfer penalties a property of the graph itself.

4. Algorithms

Dijkstra is our reference. Standard implementation with a binary heap (heapq), $O((V + E) \log V)$. It guarantees the optimum and explores nodes in increasing order of distance from the source — meaning on a large graph it expands a lot of nodes that go in the *wrong direction* from the target.

A* adds a heuristic $h(n)$ that estimates the remaining cost from n to the target. Priority becomes $g(n) + h(n)$ instead of just $g(n)$. For it to return optimal paths the heuristic must be admissible — never overestimate.

5. How each course topic actually shows up

- **BFS/DFS** — used during graph construction to verify the metro network is one connected component (we should find exactly one big SCC; any orphan is a parser bug).
- **Topological sort** — not directly applicable to the undirected route-space graph, but if we model the *time-expanded* graph (one node per (stop, line, departure_time)) as a DAG for timetable routing, topological order becomes the natural traversal.
- **MST** — useful as a sanity-check visualization: drawing the MST of the metro graph gives a clean "skeleton" of the network and is a nice contrast figure to the full graph.
- **Shortest paths (Dijkstra)** — the workhorse.
- **Advanced shortest paths (A*, possibly bidirectional Dijkstra)** — the main comparison.

6. Open questions we'll try to answer

These are the experimental questions the report can hang itself on:

- How much does the transfer penalty (1, 3, 5, 10 minutes) change the routes chosen? Are there OD pairs where the "best" route flips entirely as we vary the penalty?
- What max_speed parameter gives the strongest admissible heuristic? Is there a sweet spot, and does it depend on whether we're metro-only or include RER?
- For which queries is A* most effective vs. barely better than Dijkstra? Is there a clean relationship to great-circle distance between source and target?